YieldSense AI

Complete Code Logic, Datasets & Project Overview

The system has two distinct layers: a trained **machine-learning model** (XGBoost) that predicts crop yield, and a **rule-based logic layer** (plain if/else thresholds, no ML) that produces the risk level and recommendations. This document covers exactly what dataset the model was trained on, the full logic of both layers, and the other key facts about the project (tech stack, API endpoints, files, and current model accuracy).

0. Project at a Glance

Item	Detail
Project name	YieldSense AI
Purpose	Predicts crop yield (t/ha) from region, crop, weather, and soil inputs
ML algorithm	XGBoost Regressor (gradient-boosted decision trees), 400 trees
Fallback algorithm	scikit-learn GradientBoostingRegressor, if XGBoost isn't installed
Dataset currently used	Synthetic — 4,000 generated rows (see Section 1)
Model accuracy (on synthetic test data)	R^2 = 0.988, MAE = 0.78 t/ha, RMSE = 1.62 t/ha
Backend framework	FastAPI (Python)
Frontend (current)	Single-file HTML + vanilla JavaScript dashboard
Frontend (planned)	React.js / Next.js
Database	None yet — planned: PostgreSQL & MongoDB
Deployment	Currently hosted on Render; Docker + AWS/Azure planned

1. Dataset Used

1.1 — The dataset the model is actually trained on

No real-world agricultural dataset has been used yet. The project runs on a **synthetic (artificially generated) dataset**, produced entirely by the script **data/generate_sample_data.py** and saved as **data/sample_crop_yield_data.csv**. This was a deliberate choice to get the full pipeline — cleaning, training, API, dashboard — working end-to-end before wiring in real data.

- **Size:** 4,000 rows total (3,140 used for training, 785 used for testing after cleaning — see Section 0 table).
- **Generation method:** every row is randomly sampled from realistic statistical distributions (e.g. rainfall from a normal distribution centred at 900mm), then the yield for that row is computed with a formula rather than being real recorded data.

- **Reproducibility:** a fixed random seed (42) is used, so regenerating the script produces the exact same 4,000 rows every time.

1.2 — Every column in the dataset

Column	Type	Meaning
region	Category	North / South / East / West / Central
crop	Category	Wheat / Rice / Maize / Soybean / Cotton / Sugarcane
year	Number	Random year, 2010–2024
rainfall_mm	Number	Rainfall in millimetres
avg_temperature_c	Number	Average temperature in °C
humidity_pct	Number	Humidity percentage
soil_ph	Number	Soil pH (acidity/alkalinity)
soil_nitrogen_kg_ha	Number	Nitrogen in the soil (kg per hectare)
soil_phosphorus_kg_ha	Number	Phosphorus in the soil (kg per hectare)
soil_potassium_kg_ha	Number	Potassium in the soil (kg per hectare)
irrigation_pct_area	Number	Percentage of farm area that is irrigated
fertilizer_kg_ha	Number	Fertilizer applied (kg per hectare)
farm_area_ha	Number	Total farm area in hectares
yield_tonnes_per_ha	Number (TARGET)	The value the model is trained to predict

1.3 — How the target column (yield) is generated

Each crop has a fixed **base yield** in tonnes/ha: Wheat 3.2, Rice 4.0, Maize 5.5, Soybean 2.6, Cotton 1.8, Sugarcane 65.0. The final yield for a row is that base yield multiplied by five "effect" factors, each shaped like an inverted-V around an ideal value, plus 8% random noise so it isn't perfectly predictable:

```
rainfall_effect = 1 - |rainfall - 1000| / 3000
temp_effect = 1 - |temperature - 24| / 60
ph_effect = 1 - |soil_pH - 6.5| / 8
fert_effect = 0.7 + 0.3 * min(fertilizer / 250, 1.3)
irrigation_effect = 0.85 + 0.15 * (irrigation_pct / 100)

yield = base_yield * rainfall_effect * temp_effect * ph_effect
* fert_effect * irrigation_effect * noise
```

This gives the dataset real, learnable structure rather than pure randomness — which is exactly why the model can reach a very high R^2 (0.988) on it. That score reflects how well the model learned this known formula, not how accurate it would be on messy, real farm data.

1.4 — Real-world datasets planned to replace this (not yet integrated)

The README documents three real, publicly available data sources intended to replace the synthetic data as the next major step:

Source	What it provides	Access
FAOSTAT Crop Production (faostat.fao.org)	Country/regional crop yield, area harvested, production by crop and year	Free CSV bulk download
USDA NASS Quick Stats (quickstats.nass.usda.gov)	US county/state-level yield, acreage, and production	Free, with a public API
Kaggle "Crop Yield Prediction Dataset"	Combined crop, rainfall, temperature, soil, and yield data from multiple sources	Free download — closest match to this project's s

To switch to real data: download one of these as a CSV, map its columns to match the names in data_preprocessing.py (or edit that file's column lists to match the new data), then re-run **python model_training.py --data path/to/real_data.csv**. Expect the R^2 to drop from 0.988 — that's normal, since real agricultural data is noisier than a clean synthetic formula.

2. Preprocessing Pipeline

data_preprocessing.py runs four stages, shared identically by both training and live prediction so the model always sees data in the same shape.

a) Cleaning

- Missing numeric values → filled with the column's **median**.
- Missing categorical values → filled with the column's **mode** (most frequent).
- Extreme outliers trimmed: rows outside the 0.5th–99.5th percentile of yield are dropped.

b) Feature Engineering

- **npk_total** = nitrogen + phosphorus + potassium — a single overall soil-fertility number.
- **rainfall_temp_ratio** = rainfall ÷ temperature — a combined climate signal.

c) Categorical Encoding

Text columns (region, crop) can't feed a math model, so scikit-learn's **LabelEncoder** converts each category to an integer. During training, a fresh encoder is fitted on whatever categories exist. At prediction time, the already-fitted encoder is reused — an unseen category silently falls back to the encoder's first known class rather than crashing the API.

d) Assembly

All stages are chained by preprocess(), which returns the feature matrix X, the target y, the fitted encoders, and the exact ordered list of 15 feature columns the model expects.

3. Model Training — XGBoost (Gradient Boosted Trees)

build_model() first tries to import XGBoost; if unavailable, it falls back to scikit-learn's GradientBoostingRegressor. Both are ensembles of decision trees trained sequentially — this is the core "AI" of the project.

How gradient boosting works

- Tree 1 is built and makes a rough first prediction.
- The residual errors (actual – predicted) are computed for every row.

- Tree 2 is trained to predict those residuals — i.e. to correct tree 1's mistakes.
- A small fraction of tree 2's output (the learning rate) is added to the running prediction.
- This repeats for 400 trees total, each correcting the accumulated error of all trees before it.
- Each tree is capped at depth 6, so no single tree overfits by memorising rows.
- Each tree trains on a random 90% of rows and 90% of columns (subsampling) — similar in spirit to how Random Forests use bagging for robustness.

```
XGBRegressor(
n_estimators=400,
max_depth=6,
learning_rate=0.05,
subsample=0.9,
colsample_bytree=0.9,
objective='reg:squarederror'
)
```

Training and evaluation steps

- Data is split 80% training / 20% test (random_state=42 for reproducibility).
- The model is fit on the training set.
- Predictions on the held-out 20% are clipped so no prediction goes below 0.
- **MAE** (Mean Absolute Error) — average size of the prediction error, in t/ha.
- **RMSE** (Root Mean Squared Error) — similar to MAE but penalises large errors more.
- **R^2 (R-squared)** — the fraction of yield variance the model explains (1.0 = perfect).
- Four artifacts are saved to /models: the trained model, the fitted encoders, the feature column order, and the metrics — this becomes the "brain" the API loads.

4. Live Prediction Logic (/api/predict)

This replays the exact same transformation pipeline used in training, but on a single incoming row, so the model always sees data in the shape it learned from.

- Convert the incoming JSON request into a 1-row table.
- Run engineer_features() → adds npk_total and rainfall_temp_ratio.
- Encode region/crop using the saved encoders (unseen category → falls back to first known class).
- Reorder columns to exactly match feature_cols.json — tree models require the same column order they were trained on.
- Call model.predict(row) to get the raw prediction.
- Clip any negative prediction to 0.
- Multiply by farm_area_ha to get estimated total production.
- Run the rule-based risk scorer and recommendation generator on the raw inputs (not on the model's output).
- Return predicted yield, total production, risk level, recommendations, and model metrics as one JSON response.

5. Risk-Level Logic — Pure Rules, No ML

A simple point-counting system over four danger conditions. Nothing here is learned — it's a fixed checklist.

```
score = 0
+1 if rainfall_mm < 500 or rainfall_mm > 1800
+1 if avg_temperature_c < 10 or avg_temperature_c > 35
+1 if soil_ph < 5.0 or soil_ph > 8.5
+1 if irrigation_pct_area < 20

score 0-1 -> "Low"
score 2 -> "Moderate"
score 3-4 -> "High"
```

6. Recommendation Logic — Pure Rules, No ML

A sequential checklist; every condition that is true appends one tip to the response list.

- soil_ph < 6.0 → suggest liming to raise pH.
- soil_ph > 7.0 → suggest sulfur/acid amendments to lower pH.
- rainfall_mm < 700 → suggest supplemental irrigation.
- rainfall_mm > 1300 → suggest improving field drainage.
- irrigation_pct_area < 40 → suggest expanding irrigation coverage.
- fertilizer_kg_ha < 100 → flag a relatively low fertilizer rate.
- soil_nitrogen_kg_ha < 80 → suggest split-application of nitrogen.
- If none of the above trigger → fallback message: "Current growth conditions are well-balanced."

7. Weather & Soil Analysis Endpoints — Also Pure Rules

- **/api/weather-analysis**: classifies rainfall as Below ideal / Ideal / Above ideal against a 700–1300mm band, and temperature as Too cold (<15°C) / Favorable / Too hot (>32°C).
- **/api/soil-analysis**: classifies pH as Acidic / Optimal / Alkaline against 6.0–7.0, and rates fertility as Low / Moderate / High based on total NPK (<150 / <300 / ≥300).

8. Frontend Logic (index.html)

Plain client-side JavaScript, no framework.

- Collects all 13 form fields into a JSON object on submit.
- Sends a POST request to /api/predict on the same origin — works both locally and when deployed, since the backend also serves this file as static content.
- On success, injects the returned yield, total production, risk level, and recommendations into the results panel as HTML.
- On failure, shows an error message with a hint to check the backend is running.

9. Project File Structure

File	Purpose
data/generate_sample_data.py	Generates the 4,000-row synthetic dataset
data/sample_crop_yield_data.csv	The generated dataset itself
backend/data_preprocessing.py	Cleaning, feature engineering, and encoding — shared by training and prediction
backend/model_training.py	Trains the XGBoost model and saves all model artifacts
backend/main.py	FastAPI server — exposes /api/predict, /api/weather-analysis, /api/soil-analysis
backend/requirements.txt	Python dependencies (fastapi, xgboost, scikit-learn, pandas, etc.)
frontend/index.html	The single-file web dashboard (HTML + CSS + vanilla JS)
models/yield_model.joblib	The trained model, saved for reuse
models/encoders.joblib	The fitted LabelEncoders for region/crop
models/feature_cols.json	The exact ordered list of features the model expects
models/metrics.json	Saved accuracy metrics from the last training run

10. API Endpoints

Endpoint	Method	What it does
/api/status	GET	Health check — confirms the model is loaded and returns its saved metrics
/api/predict	POST	Main endpoint — takes farm/weather/soil inputs, returns predicted yield, total production, r
/api/weather-analysis	POST	Rule-based check of whether rainfall/temperature are in the ideal growing range
/api/soil-analysis	POST	Rule-based check of soil pH status and overall NPK fertility rating

11. Tech Stack — What's Built vs. What's Planned

It's important to be clear about this gap, especially for a demo or viva: the presentation's tech-stack slide lists the full intended stack, but only part of it is actually implemented right now.

Layer	Currently built	Planned (not yet built)
Backend	Python + FastAPI	—
ML	scikit-learn + XGBoost	TensorFlow (listed in stack, unused so far)
Frontend	Plain HTML/CSS/JS single file	React.js / Next.js
Database	None — no persistence of past predictions	PostgreSQL & MongoDB

Auth	None — API is fully open	JWT-based auth with role-based access
Deployment	Hosted directly on Render	Docker containerization, AWS / Azure

End-to-End Summary

Generate synthetic data → clean and engineer features → encode categories → train 400 boosted decision trees to approximate the yield formula → save the model → at request time, replay the same transformations on one row → get a yield prediction from the trees → layer deterministic rule-based risk and recommendation logic on top → serve it all through a FastAPI JSON API → render it in a plain HTML/JS dashboard.

12. Techniques Used — Quick Reference

Technique	Where it's used
Gradient Boosted Trees (XGBoost)	Core yield prediction — supervised regression
Label Encoding	Converting region/crop text into numbers
Feature engineering	npk_total, rainfall_temp_ratio
Train/test split (80/20)	Model evaluation on held-out data
Regression metrics: MAE, RMSE, R^2	Measuring prediction accuracy
Rule-based expert system (if/else)	Risk level + recommendations — not ML
REST API model serving (FastAPI)	Exposes the trained model as /api/predict